



RenkerVault

Wie es funktioniert

Eine Erklärung in zwei Teilen: verständlich für alle — und im Detail für Informatiker*innen (angewandte Informatik).

RENKER INDUSTRIES — SECURE COMMS DIVISION

Stand: Prototyp v0.1 · Dieses Dokument beschreibt den aktuellen Entwicklungsstand.

Inhalt

Teil A — Für Laien

1. Was ist RenkerVault?
2. Warum es RenkerVault gibt: der Hintergrund („Chat-Kontrolle“)
3. Das Grundprinzip: der doppelt verschlossene Umschlag
4. Was RenkerVault besonders macht: das Sicherheitssystem
5. Funktionen im Überblick
6. So benutzt man RenkerVault
7. Mit anderen chatten: einen eigenen Relay betreiben
8. Schutz vor Quantencomputern & maximale Anonymität
9. Wo RenkerVault an seine Grenzen kommt

Teil B — Für Informatiker (angewandte Informatik)

10. Architektur-Überblick
11. Identität & Schlüsselmaterial
12. Initialer Schlüsselaustausch (X3DH-lite, Hybrid Post-Quantum)
13. Double Ratchet im Detail
14. Gruppen: Sender-Keys-lite
15. Envelope-Format & Metadaten
16. Lokale Verschlüsselung (At-Rest)
17. Code-Architektur
18. Bekannte Grenzen des Prototyps
19. Deployment: Desktop, Mobile, Web
20. Hosting & TLS im Detail
21. Server-Hardening gegen Missbrauch
22. Tor Hidden Service & „Sitzung verbrennen“

Glossar

Für Laien

Verständlich erklärt — kein Vorwissen nötig

1. Was ist RenkerVault?

RenkerVault ist eine Chat-App — vergleichbar mit Signal, Telegram oder WhatsApp. Man kann damit mit einzelnen Personen oder in Gruppen chatten, Bilder und Sprachnachrichten schicken. Der entscheidende Unterschied: RenkerVault ist speziell für Menschen gebaut, die maximale Privatsphäre wollen und sich gegen anlasslose Chat-Überwachung absichern möchten.

Das zentrale Versprechen: **Selbst der Betreiber des Servers kann die Nachrichten nicht mitlesen.** Das nennt man „Zero-Knowledge“ — der Server transportiert nur verschlüsselte Daten, ohne zu wissen, was darin steht.

Zusätzlich hat RenkerVault ein eigenes **Einbruchsalarm-System**: Es erkennt, wenn jemand versucht, sich unbefugt Zugang zu verschaffen, und schlägt sofort sichtbar Alarm.

2. Warum es RenkerVault gibt: der Hintergrund („Chat-Kontrolle“)

Ein Bericht der Europäischen Kommission (2025) über eine bestehende EU-Verordnung zeigt, wie viel bereits heute **freiwillig** von großen Anbietern (Google, Meta, Microsoft u. a.) automatisiert durchsucht wird: Microsoft verarbeitete 2024 weltweit 9,6 Milliarden Inhalte, Meta stufte 2023 3,6 Millionen Inhalte als illegal ein — bei Fehlerquoten zwischen 0,1 % und über 1 %. Bei diesen Größenordnungen bedeutet selbst eine kleine Fehlerquote, dass tausende komplett unbeteiligte Menschen fälschlich markiert werden, Konten gesperrt bekommen oder ihre Daten an Dritte gemeldet werden.

Ein noch nicht beschlossener Folge-Vorschlag der EU-Kommission — umgangssprachlich „Chat-Kontrolle“ genannt — würde ein solches Scannen für **alle** Chat-Anbieter verpflichtend machen, auch für Ende-zu-Ende-verschlüsselte Dienste. Technisch ginge das nur über sogenanntes **Client-Side-Scanning**: Der Inhalt würde auf dem eigenen Gerät analysiert, bevor er verschlüsselt bzw. nachdem er entschlüsselt wurde. Damit entstünde ein struktureller Zugriffspunkt auf jeden privaten Chat — unabhängig davon, wie gut die Übertragung selbst gesichert ist.

RenkerVault ist bewusst so gebaut, dass ein solcher Zugriffspunkt gar nicht existiert: Der Relay-Server bekommt nie Klartext zu sehen und kann daher auch nicht nachträglich zu einem Scanner umgebaut werden. Das macht RenkerVault zu keinem Werkzeug „gegen“ Kinderschutz, sondern zu derselben Art von Technologie wie Signal, Session oder Threema: Schutz der Privatsphäre durch Architektur, statt durch Vertrauen in eine Stelle, die jederzeit ausgeweitet oder missbraucht werden könnte.

3. Das Grundprinzip: der doppelt verschlossene Umschlag

Stell dir vor, du willst deinem Freund einen Brief schicken, ohne dass die Post ihn lesen kann. Du steckst den Brief in eine kleine Kiste und schließt sie mit einem Vorhängeschloss ab, dessen Schlüssel nur dein Freund besitzt. Egal wer die Kiste unterwegs in die Hand bekommt — ohne den passenden Schlüssel bleibt sie zu.

Genau das passiert bei jeder Nachricht in RenkerVault, nur eben mit Mathematik statt mit einem echten Schloss: Dein Gerät verschlüsselt die Nachricht mit einem Schlüssel, den nur das Gerät deines Kontakts entschlüsseln kann. Der Server transportiert nur die verschlossene „Kiste“.

Ein Extra-Trick macht es noch sicherer: Nach jeder einzelnen Nachricht wird ein neues Schloss verwendet — wie ein Zahlenschloss, das sich nach jedem Öffnen automatisch neu einstellt. Das nennt man in der Fachsprache „Double Ratchet“ (deutsch etwa: „doppeltes Schaltwerk“). Der Vorteil: Sollte jemand irgendwann doch an einen einzelnen Schlüssel gelangen, kann er damit trotzdem nur genau eine Nachricht lesen — nicht den gesamten bisherigen oder zukünftigen Chatverlauf.

„Selbst wenn ein Schlüssel verloren geht, bleiben alle anderen Nachrichten geschützt — vorher genauso wie nachher.“

Bevor man einem neuen Kontakt vertraut, kann man außerdem eine sogenannte **Safety Number** vergleichen — eine 60-stellige Zahlenfolge (oder ein QR-Code), die bei beiden Seiten exakt gleich sein muss. Stimmt sie überein, kann sich niemand unbemerkt dazwischenschalten.

4. Was RenkerVault besonders macht: das Sicherheitssystem

- **Einbruchserkennung:** Werden fünfmal hintereinander falsche Passwörter eingegeben, sperrt sich das Konto für 60 Sekunden und ein deutlich sichtbarer roter Alarm erscheint.
- **Neue-Geräte-Warnung:** Wird versucht, das Konto von einem unbekannten Gerät aus zu nutzen, muss das erst von einem bereits vertrauten Gerät manuell bestätigt werden.
- **Manipulationserkennung:** Die lokal gespeicherten Daten werden mit einer Prüfsumme abgesichert. Wurde daran manipuliert, sperrt sich die App automatisch und zeigt nur noch den Alarm-Bildschirm.
- **Notfall-PIN (Duress-Modus):** Wird man gezwungen, die App zu entsperren, kann man stattdessen eine geheime Notfall-PIN eingeben. Es öffnet sich dann eine leere, harmlos aussehende Ansicht — die echten Daten bleiben verborgen und unangetastet.
- **Security-Log:** Jedes sicherheitsrelevante Ereignis wird mit Uhrzeit protokolliert und ist jederzeit einsehbar.

5. Funktionen im Überblick

Bereich	Was es kann
Einzelchats & Gruppen	Ende-zu-Ende-verschlüsselte Nachrichten, Bilder, Sprachnachrichten

Bereich	Was es kann
Antworten & Reaktionen	Auf Nachrichten antworten/zitieren, bearbeiten, löschen, weiterleiten, mit Emoji reagieren
Verswindende Nachrichten	Nachrichten löschen sich nach einer einstellbaren Zeit automatisch
Kein Klarname nötig	Konto entsteht aus einer zufälligen ID + selbstgewähltem Passwort — keine Telefonnummer, keine E-Mail
Geräteverwaltung	Übersicht aller angemeldeten Geräte, einzelne sofort remote abmelden
Organisation	Chats anpinnen, stummschalten, archivieren, durchsuchen
Präsenz	Sehen, ob ein Kontakt gerade online ist bzw. wann er zuletzt aktiv war

6. So benutzt man RenkerVault

- **Tresor erstellen:** Beim ersten Start wird automatisch eine Konto-ID erzeugt. Man vergibt ein Passwort (die „Passphrase“) und optional eine Notfall-PIN.
- **Kontakt hinzufügen:** Man tauscht mit einer anderen Person die eigene Konto-ID aus (z. B. mündlich oder über einen anderen Kanal) und trägt sie unter „Kontakte“ ein. Die App baut dann automatisch eine gesicherte Verbindung auf.
- **Chatten & Gruppen:** Danach funktioniert alles wie in jeder anderen Chat-App — nur eben mit dem beschriebenen Sicherheitsnetz im Hintergrund.
- **Sicherheitszentrale:** Hier sieht man alle eigenen Geräte, kann die Datenbank auf Manipulation prüfen und Schlüssel manuell erneuern.

7. Mit anderen chatten: einen eigenen Relay betreiben

Damit RenkerVault mit anderen Personen funktioniert, braucht es einen „Relay“ — eine Art digitalen Briefkasten-Verteiler, der die verschlossenen „Kisten“ (siehe Abschnitt 3) zwischen den Geräten hin- und herträgt, ohne sie öffnen zu können. Zwei Möglichkeiten, das einzurichten:

- **Einen kleinen Server mieten (empfohlen):** Kostet bei den meisten Anbietern nur wenige Euro im Monat, läuft rund um die Uhr, hat eine feste Internetadresse. Eine Schritt-für-Schritt-Anleitung dafür liegt dem Projekt bei (`deploy/DEPLOYMENT.md`).
- **Vom eigenen PC aus:** Ebenfalls möglich, aber mit spürbaren Nachteilen — der PC muss dafür eingeschaltet und mit dem Internet verbunden bleiben, und im Heim-Router muss eine „Weiterleitung“ eingerichtet werden, wodurch der PC direkt aus dem Internet erreichbar wird. Für kurzes Ausprobieren im Familien-/Freundeskreis in Ordnung, für dauerhaften Betrieb wird ein gemieteter Server empfohlen.

In beiden Fällen wird die Adresse des Relays anschließend nur einmal in den **Einstellungen** der App eingetragen — alle, die miteinander chatten wollen, tragen dieselbe Adresse ein. Wichtig: Die Adresse muss mit `wss://` beginnen (nicht `ws://`), sobald es sich nicht mehr um den eigenen Rechner handelt — sonst ist die Verbindung zum Server selbst nicht zusätzlich abgesichert. Die App warnt davor automatisch.

8. Schutz vor Quantencomputern & maximale Anonymität

Zukünftige, ausreichend große Quantencomputer könnten heutige Verschlüsselungsverfahren wie X25519 brechen. Das Problem ist dabei nicht erst die Zukunft: Ein Angreifer kann schon **heute** verschlüsselten Datenverkehr aufzeichnen und ihn einfach so lange aufbewahren, bis ein passender Quantencomputer existiert — man nennt das „**Harvest Now, Decrypt Later**“ („jetzt sammeln, später entschlüsseln“). Deshalb reicht es nicht, sich erst dann um Quantensicherheit zu kümmern, wenn die Technik wirklich da ist.

RenkerVault begegnet dem, indem der allererste Schlüsselaustausch mit einem Kontakt zusätzlich zum bisherigen Verfahren ein zweites, als quantenresistent geltendes Verfahren nutzt (**ML-KEM-768**, genormt als FIPS 203) — beide Ergebnisse werden zu einem gemeinsamen Schlüssel vermischt. Das bedeutet: Selbst wenn eines der beiden Verfahren in Zukunft gebrochen werden sollte, bleibt der Erstkontakt trotzdem geschützt, solange das jeweils andere Verfahren hält. Genau diesen Ansatz verwendet auch Signal (dort „PQXDH“ genannt).

Für mehr Anonymität (niemand soll sehen können, wer mit wem chattet) lässt sich der Relay-Server zusätzlich ausschließlich über einen **Tor Hidden Service** betreiben — dann hat er gar keine gewöhnliche Internetadresse mehr, sondern nur eine „onion“-Adresse, die ausschließlich über das Tor-Netzwerk erreichbar ist. Damit lässt sich weder von außen noch vom Betreiber der Leitung feststellen, wer sich mit dem Relay verbindet.

Außerdem gibt es einen ■-Knopf („Sitzung verbrennen“): Ein Klick mit Bestätigung löscht einen kompletten Chatverlauf sofort und unwiderruflich, bei echten Kontakten inklusive der gesamten Verschlüsselungssitzung. Danach ist es, als hätte die Unterhaltung nie stattgefunden — dasselbe Prinzip, das auch anonyme Werkzeuge wie „OnionShare“ verfolgen.

Zwei Dinge wurden bewusst NICHT eingebaut, trotz ihres Rufs als „ultimative“ Sicherheit — mit ehrlicher Begründung: Ein **One-Time-Pad** wäre zwar theoretisch perfekt sicher, verlangt aber vorab genau so viel Schlüsselmaterial wie später Nachrichten verschickt werden, das zudem niemals ein zweites Mal verwendet werden darf — in der Praxis extrem fehleranfällig. Und ein „echter Quanten-Zufallsgenerator“ bräuchte spezielle Hardware, die es in einer normalen App schlicht nicht gibt; der stattdessen verwendete Standard-Zufallsgenerator des Browsers ist der fachlich korrekte und anerkannte Ansatz.

9. Wo RenkerVault an seine Grenzen kommt

Ehrlichkeit gehört zu einem Sicherheitsprodukt dazu: RenkerVault ist aktuell ein **Prototyp**, kein von unabhängigen Experten geprüftes Produkt. Das bedeutet konkret:

- Die Verschlüsselung wurde selbst zusammengesetzt (aus geprüften Bausteinen) — aber die Gesamt-Komposition wurde noch nicht von externen Fachleuten kontrolliert.
- Der Server sieht zwar keine Nachrichteninhalte, aber technisch bedingt einige Metadaten (z. B. wer mit wem gerade Kontakt hat).
- Es werden bewusst **keine** automatischen Link-Vorschauen geladen, weil das selbst Daten an Dritte verraten würde.
- Für den ernsthaften Einsatz bräuchte es noch ein externes Sicherheits-Audit.

Alle technischen Details dazu stehen im Dokument SECURITY.md im Projekt.

TEIL B

Für Informatiker

Architektur, Kryptografie-Protokoll und Code-Struktur im Detail

10. Architektur-Überblick

RenkerVault besteht aus zwei unabhängigen Codebasen: einem **Client** (React + TypeScript, per Vite gebaut) und einem **Relay-Server** (Node.js + ws). Der Relay ist bewusst „dumm“ gehalten: Er kennt Konto-IDs, öffentliche Schlüssel, Geräte-Metadaten und opake, bereits verschlüsselte Envelopes — niemals Klartext.

```
[Client A] --hello/proof (Ed25519)--> [Relay] <--proof-- [Client B]
| X25519/Ed25519-Schlüssel lokal          |
| Double Ratchet je Kontakt              | speichert NUR:
| AES-256-GCM verschlüsselte Envelopes   | Konto-IDs, Pubkeys,
+----- ct (Ciphertext) ----->| Geräte, Envelopes
                                   | (Weiterleitung/Queue)
```

Authentifizierung läuft passwortlos über eine Ed25519-Challenge-Response: Der Relay schickt eine Zufalls-Nonce, der Client signiert sie mit seinem privaten Schlüssel — die Passphrase selbst verlässt den Client nie.

11. Identität & Schlüsselmaterail

Jede Identität besteht aus drei Schlüsselpaaren (alle X25519 bzw. Ed25519, erzeugt via @noble/curves):

Schlüssel	Typ	Zweck
Identity Key (IK)	X25519	Langzeit-Identität, Basis für Diffie-Hellman
Signed Prekey	X25519	Veröffentlicht beim Relay; ermöglicht asynchronen Erstkontakt
PQ-Prekey	ML-KEM-768	Veröffentlicht beim Relay; Post-Quantum-Anteil des Hybrid-Handshakes (siehe Abschnitt 12)
Signing Key	Ed25519	Passwortlose Authentifizierung beim Relay (Challenge-Response)

Getrennte Kurven für Verschlüsselung (X25519) und Signaturen (Ed25519) folgen der etablierten Signal-Praxis: Diffie-Hellman-Schlüssel und Signaturschlüssel sollten nie identisch sein.

12. Initialer Schlüsselaustausch (X3DH-lite, Hybrid Post-Quantum)

Beim ersten Kontakt berechnen beide Seiten ein gemeinsames Geheimnis aus zwei Diffie-Hellman-Operationen (statt der drei bis vier bei vollem X3DH) plus einer Kapselung mit dem Post-Quantum-Verfahren ML-KEM-768:

```
SK = HKDF-SHA256( DH(IK_a, IK_b) || DH(EK_a, SPK_b) || PQ_secret )

IK_a      = Identity Key des Initiators (privat) / Gegenseite (public)
EK_a      = frischer Ephemeral Key des Initiators
SPK_b      = veröffentlichter Signed Prekey der Gegenseite
PQ_secret = ML-KEM-768-Shared-Secret, siehe unten
```

Der Signed Prekey dient zugleich als initialer Double-Ratchet-Schlüssel des Responders — so kann der Initiator sofort verschlüsseln, ohne dass die Gegenseite online sein muss (asynchroner Erstkontakt). Bewusster Unterschied zu vollem X3DH: es gibt keine One-Time-Prekeys, der Signed Prekey wird wiederverwendet (wie bei Signal selbst üblich) — dokumentierter Kompromiss in SECURITY.md.

Post-Quantum-Anteil (implementiert in `crypto/pq.ts`): Der Responder veröffentlicht zusätzlich einen ML-KEM-768-Prekey. Der Initiator führt damit eine `encapsulate()`-Operation aus, die sowohl ein Chiffre (an den Responder geschickt) als auch ein Shared Secret erzeugt; der Responder gewinnt dasselbe Shared Secret per `decapsulate()` zurück. Beide Geheimnisse (klassisch + Post-Quantum) fließen gemeinsam in die HKDF ein — bricht eines der beiden Verfahren künftig, bleibt der Erstkontakt trotzdem geschützt, solange das andere hält (dieselbe „Hybrid“-Strategie wie Signals PQXDH).

Wichtige Grenze, transparent dokumentiert: Nur dieser **Erstkontakt-Handshake** ist Post-Quantum-hybrid. Die fortlaufenden Double-Ratchet-Schritte (Abschnitt 13) basieren weiterhin auf klassischem X25519-ECDH — ein vollständig quantensicherer Ratchet ist derzeit kein etablierter Industriestandard. Ebenso bewusst nicht umgesetzt: ein generisches One-Time-Pad (Schlüsselmaterial müsste die Nachrichtenmenge decken und exakt einmal verwendet werden — in der Praxis fragil) und ein „Quanten-Zufallsgenerator“ (erfordert dedizierte Hardware, in Software nicht realisierbar; `crypto.getRandomValues()` ist der korrekte CSPRNG-Standardansatz). Details: SECURITY.md, Abschnitte 4b/4d.

13. Double Ratchet im Detail

Implementiert in `crypto/ratchet.ts`, komponiert aus auditierten Primitiven (kein Custom-Krypto-Primitiv):

```
KDF_RK(rootKey, dhOut) = HKDF-SHA256(dhOut, salt=rootKey) -> (rootKey', chainKey)
KDF_CK(chainKey)       = ( HMAC(chainKey, 0x02), HMAC(chainKey, 0x01) )
                        naechste Kette      Message Key

encrypt(plaintext):    mk = KDF_CK-Schritt; ct = AES-256-GCM(mk, plaintext, aad=header)
decrypt(msg):          DH-Ratchet-Schritt falls neuer Sender-Public-Key im Header,
                      dann KDF_CK-Schritte bis zur passenden Nachrichten-Nummer
```

- **Perfect Forward Secrecy:** Jeder Message Key wird sofort nach Gebrauch verworfen — Kompromittierung eines Keys entschlüsselt nur eine einzige Nachricht.
- **Post-Compromise Security:** Mit jeder Antwort der Gegenseite wird per neuem Diffie-Hellman-Schritt ein frischer Root Key etabliert — die Sitzung „heilt“ sich selbst.
- **Out-of-Order-Zustellung:** Übersprungene Message Keys werden (begrenzt auf 64 Einträge) zwischengespeichert, damit später eintreffende Nachrichten trotzdem entschlüsselbar bleiben.

14. Gruppen: Sender-Keys-lite

Gruppen nutzen keinen Double Ratchet pro Nachricht (das würde bei n Mitgliedern n parallele Sitzungen erfordern), sondern einen gemeinsamen **Epoch-Key** (256-Bit-Zufallswert, AES-256-GCM). Der Owner erzeugt ihn und verteilt ihn **individuell über die bestehenden 1:1-Ratchet-Kanäle** an jedes Mitglied — die Gruppenschlüssel-Verteilung selbst ist also Ende-zu-Ende-verschlüsselt.

Bei jeder Mitgliederänderung (hinzufügen/entfernen) wird eine neue Epoche erzeugt und erneut verteilt: entfernte Mitglieder besitzen den neuen Schlüssel nicht und können spätere Nachrichten nicht mehr lesen. Reguläre Nachrichten werden nur **einmal** mit dem aktuellen Epoch-Key verschlüsselt und dasselbe Chifftrat an jedes Mitglied einzeln adressiert (der Relay kennt kein natives Gruppen-Fan-out).

15. Envelope-Format & Metadaten

Das Transportformat, das der Client an den Relay sendet (Auszug aus `net/client.ts`):

```
interface Envelope {
  ct: string; // Base64-Chiffretext (AES-256-GCM)
  chatId, chatKind, kind, // Routing-Metadaten (fuer Relay sichtbar)
  msgId, ts, fromName
  header?: RatchetHeader; // 1:1: dh-Pubkey, Kettenposition
  epoch?: number; // Gruppe: aktuelle Schluesel-Epoche
  x3dh?: { ephPub, identityPub }; // nur allererste 1:1-Nachricht
  replyTo?, forwardedFrom?, targetMsgId?, emoji? // Anwendungs-Events
}
```

Wichtig für die Bewertung des Vertrauensmodells: Felder außerhalb von `ct` sind für den Relay **sichtbar** (Routing-Metadaten, analog zu Signal). Antwort-Zitate und der „Weitergeleitet von“-Name laufen aktuell ebenfalls als Metadaten, nicht im Chifftrat — ein bewusst dokumentierter Kompromiss (siehe SECURITY.md, Abschnitt 3).

16. Lokale Verschlüsselung (At-Rest)

```
Passphrase --Argon2id(64 MiB, t=4)--> KEK
KEK --AES-256-GCM--> wrappt zufaelligen Master-Key (32 Byte)
Master-Key --AES-256-GCM--> verschluesselt kompletten Vault-Zustand
HKDF(Master-Key, "mac") --> MAC-Key
MAC-Key --HMAC-SHA256--> ueber den Daten-Ciphertext (Manipulationserkennung)
```

Die HMAC-Prüfung läuft bei jedem Entsperren und auf Abruf. Schlägt sie fehl, wird ein Alarm ausgelöst und optional ein Auto-Lockdown erzwungen (nur der Alarm-Bildschirm bleibt sichtbar). Ein separater Argon2id-Hash für eine optionale Duress-PIN öffnet stattdessen eine leere Fake-Ansicht, ohne den echten Master-Key preiszugeben.

17. Code-Architektur

Pfad	Verantwortlichkeit
crypto/primitives.ts	Duene Wrapper um @noble/curves, @noble/hashes, hash-wasm (Argon2id), WebCrypto (AES-GCM)
crypto/ratchet.ts	Double Ratchet + X3DH-lite-Handshake (Protokoll-Komposition)
crypto/vault.ts	At-Rest-Verschlüsselung, HMAC-Integrität, Duress-Modus
crypto/safety.ts	Safety Numbers / Fingerprints
net/client.ts	WebSocket-Client, Envelope-Zustellung, Kontakt-Lookup
net/realchat.ts	Sitzungs-/Gruppenschlüssel-Engine fuer echte Kontakte (persistiert)
demo/seed.ts	Vorfuehr-Welt mit simulierten, aber ECHT verschlüsselten Peers
state/types.ts	Zentrale TypeScript-Typen (Chat, Message, Identity, VaultData, ...)
ui/*.tsx	React-Komponenten (HUD-Dashboard, Chatfenster, Sicherheitszentrale)
server/src/index.js	Zero-Knowledge-Relay: Auth, Rate-Limiting, Envelope-Routing/Queue

18. Bekannte Grenzen des Prototyps

- Double-Ratchet- und Vault-**Komposition** ist kryptografisch begründet, aber nicht extern auditiert (die Primitive selbst sind es).
- Kein volles X3DH (keine One-Time-Prekeys) — etwas schwächere Asynchronitäts-Garantien.
- Relay hält Zustand nur im RAM (Prototyp); Produktion bräuchte PostgreSQL + persistente Queues.
- TLS (wss://) ist jetzt verfügbar (nativ oder per Reverse-Proxy), aber optional — wird beim Verbinden zu einem entfernten, nicht-lokalen Host über ws:// aktiv per Warnung angemahnt.
- Metadaten (wer-mit-wem-wann) sind für den Relay sichtbar; Sealed-Sender-artige Techniken fehlen.
- Web-Auslieferung kann durch einen kompromittierten Server manipuliert werden — echte Sicherheit braucht signierte Desktop-/Mobile-Builds.
- Rate-Limiting/Verbindungsdeckel schützen vor einzelnen Angreifern, nicht vor verteilten Angriffen (DDoS) aus vielen IP-Adressen gleichzeitig.

Vollständige, laufend gepflegte Liste: SECURITY.md im Projekt-Root.

19. Deployment: Desktop, Mobile, Web

Plattform	Technologie	Kurzbeschreibung
Web	Vite Dev-/Prod-Build	Dieselbe Codebasis läuft direkt im Browser
Desktop (Windows)	Tauri (Rust + WebView2)	Native Shell um dieselbe React-App; kleinerer Angriffsvektor als Electron, da kein Node-Kontext im Renderer

Plattform	Technologie	Kurzbeschreibung
Installer	Inno Setup	Erzeugt Setup.exe (Desktop-/Startmenü-Verknüpfung, optional Relay-Server-Bundle)
Mobile (Android)	Capacitor	Wrapped dieselbe Web-Codebasis 1:1 in eine native WebView-App — kein Code-Zweitschreiben; ohne Google-Play-Services/Firebase-Abhängigkeit, läuft daher auch auf GrapheneOS
Relay-Server	Node.js (eigenständig)	Adresse in den Einstellungen konfigurierbar (ws:// lokal, wss:// produktiv)

Diese Mehrfach-Verwertung derselben TypeScript-Codebasis (Web/Desktop/Mobile) ist möglich, weil sämtliche Kryptografie rein clientseitig in JavaScript/WebCrypto implementiert ist — es gibt keine plattformspezifische Logik, die separat portiert werden müsste.

20. Hosting & TLS im Detail

Der Relay unterstützt zwei TLS-Betriebsarten. Empfohlen ist ein **Reverse-Proxy** (Caddy) auf demselben Host, der automatisch ein Let's-Encrypt-Zertifikat bezieht und erneuert und WebSocket-Verkehr transparent weiterleitet — der Node-Prozess selbst lauscht dabei nur auf `127.0.0.1` und ist vom Internet aus nicht direkt erreichbar:

```
Internet --443/tcp--> [Caddy: TLS-Termination]
                    | reverse_proxy 127.0.0.1:8787
                    v
                [Node-Relay auf localhost, kein Internet-Zugriff]
```

Alternativ terminiert der Relay TLS **nativ** selbst, über die Umgebungsvariablen `TLS_CERT_FILE/TLS_KEY_FILE` (dann erzeugt der Prozess einen `https.Server` statt `http.Server` und der `WebSocketServer` wird darüber gemountet). Nachteil: Zertifikaterneuerung (z. B. per `certbot`) muss manuell organisiert werden, während Caddy das automatisch erledigt.

Client-IP-Ermittlung hinter einem Reverse-Proxy läuft über `X-Forwarded-For`, aber **nur** wenn `TRUST_PROXY=1` explizit gesetzt ist — sonst könnten Clients diesen Header selbst gefälscht mitschicken und so Rate-Limits/Verbindungsdeckel umgehen. Vollständige Schritt-für-Schritt-Anleitung inkl. Firewall- und `systemd`-Konfiguration: `deploy/DEPLOYMENT.md`.

21. Server-Hardening gegen Missbrauch

- **Verbindungsdeckel pro IP** (Standard: 20 gleichzeitige Sockets) — begrenzt trivialen Ressourcenverbrauch durch eine einzelne Quelle.
- **Auth-Timeout** (15 s): Sockets, die sich nie authentifizieren, werden automatisch getrennt statt unbegrenzt offenzustehen.
- **Argon2id auf 64 MiB / 4 Iterationen** angehoben (vorher 32 MiB / 3) — mehr Rechenaufwand pro Entsperrversuch, erschwert Offline-Brute-Force auf ein gestohlenes Vault-Backup spürbar, bleibt aber im von OWASP empfohlenen Bereich für interaktive Anwendungen.

- **systemd-Sandbox** für den Produktivbetrieb (siehe `deploy/renkervault-relay.service`): eigener unprivilegierter Nutzer, `ProtectSystem=strict`, `NoNewPrivileges`, keine Capabilities.

Ausdrücklich **nicht** abgedeckt: verteilte Angriffe (DDoS) aus vielen IP-Adressen gleichzeitig — dafür ist ein vorgelagertes CDN/DDoS-Schutzangebot nötig, das außerhalb des Scopes dieses Prototyps liegt (siehe `SECURITY.md`, Punkt 12).

22. Tor Hidden Service & „Sitzung verbrennen“

Als dritte Hosting-Option (neben nativem TLS und Reverse-Proxy, siehe Abschnitt 20) unterstützt der Relay den Betrieb als reiner **Tor Hidden Service**: `deploy/torrc.snippet` bindet den Relay-Port ausschließlich an `127.0.0.1` und veröffentlicht ihn über Tor unter einer `.onion`-Adresse.

```
HiddenServiceDir /var/lib/tor/renkervault/  
HiddenServicePort 80 127.0.0.1:8787
```

Der Unterschied zu Weg 1/2 (VPS bzw. eigener PC mit TLS): Es gibt keinen öffentlichen DNS-Namen und keinen von außen offenen Port mehr — die Verbindung läuft komplett über das Tor-Netzwerk, wodurch weder ein Netzwerk-Beobachter noch der Hosting-Anbieter selbst die reale IP der verbindenden Clients einsehen kann. TLS-Zertifikate werden dabei nicht benötigt, da Tor die Transportverschlüsselung und Standort-Verschleierung bereits selbst übernimmt (die App-eigene Ende-zu-Ende-Verschlüsselung aus Abschnitt 12/13 bleibt davon unabhängig zusätzlich bestehen).

„Sitzung verbrennen“ (implementiert in `net/realchat.ts: dropSession()`, aufgerufen aus `ui/App.tsx: handleBurnChat()`): Löscht sofort und unwiderruflich den lokalen Nachrichtenverlauf eines Chats. Bei echten 1:1-Kontakten wird zusätzlich der Ratchet-Zustand aus der Map entfernt und der Kontakt selbst gelöscht — ein späterer erneuter Kontakt erfordert zwingend einen kompletten neuen X3DH-Handshake, es gibt keinen Zustand, der wiederhergestellt werden könnte. Konzeptionell identisch zum „ephemeral by design“-Prinzip von OnionShare: nach dem Schließen existiert kein Beweis mehr, dass eine Verbindung bestand.

Glossar

Ende-zu-Ende-Verschlüsselung (E2E)

Nur Sender und Empfänger können die Nachricht lesen — kein Zwischenstation, auch nicht der Serverbetreiber.

Zero-Knowledge

Der Server speichert/verarbeitet keine Information, aus der sich der Nachrichteninhalte ableiten ließe.

Perfect Forward Secrecy (PFS)

Ein kompromittierter Schlüssel gefährdet nicht die Vertraulichkeit vergangener Nachrichten.

Post-Compromise Security

Nach einer Kompromittierung stellt das Protokoll durch neue Schlüsselvereinbarung von selbst wieder Sicherheit her.

HKDF

HMAC-based Key Derivation Function — leitet aus einem Geheimnis mehrere unabhängige Schlüssel ab.

HMAC

Keyed-Hash Message Authentication Code — beweist Integrität und Authentizität von Daten.

AES-256-GCM

Symmetrisches Verschlüsselungsverfahren mit eingebauter Authentifizierung (Manipulationserkennung).

Argon2id

Passwort-Hashing-Verfahren, das absichtlich rechenintensiv ist, um Brute-Force-Angriffe zu erschweren.

X25519 / Ed25519

Elliptische-Kurven-Verfahren für Schlüsselaustausch (X25519) bzw. digitale Signaturen (Ed25519).

Double Ratchet

Von Signal entwickeltes Protokoll, das bei jeder Nachricht automatisch neue Schlüssel erzeugt.

Prekey

Im Voraus veröffentlichter öffentlicher Schlüssel, der einen verschlüsselten Erstkontakt ohne Online-Gegenüber ermöglicht.

Sender Keys

Effizientes Gruppen-Verschlüsselungsverfahren mit einem gemeinsamen, bei Änderungen rotierenden Schlüssel.

WebView

Eingebetteter Browser-Baustein, mit dem native Apps Webinhalte anzeigen (Basis von Tauri und Capacitor).

Tauri

Framework für native Desktop-Apps mit Rust-Backend und schlankem, systemeigenem WebView statt gebündeltem Chromium.

Capacitor

Framework, das eine Web-App unverändert in eine native iOS-/Android-App einbettet.

VPS

Virtual Private Server — ein gemieteter, virtueller Server bei einem Hosting-Anbieter, oft schon für wenige Euro im Monat.

Reverse-Proxy

Ein Server, der eingehende Verbindungen entgegennimmt (inkl. TLS) und an einen dahinterliegenden Dienst weiterleitet.

Let's Encrypt

Kostenlose, automatisierte Zertifizierungsstelle für TLS-Zertifikate, u. a. von Caddy automatisch genutzt.

Caddy

Webserver/Reverse-Proxy, der TLS-Zertifikate automatisch bezieht und erneuert.

systemd

Der auf den meisten Linux-Servern verwendete Dienst-Manager, der Programme im Hintergrund startet, überwacht und bei Bedarf neu startet.

Post-Quantum-Kryptografie

Verschlüsselungsverfahren, die auch gegen ausreichend große Quantencomputer als sicher gelten (im Gegensatz zu klassischem ECDH wie X25519).

ML-KEM-768 (Kyber)

Von NIST genormtes (FIPS 203) Post-Quantum-Verfahren zur Schlüsseleinigung; in RenkerVault hybrid mit X25519 kombiniert.

Harvest Now, Decrypt Later (HNDL)

Angriffsstrategie: verschlüsselten Datenverkehr schon heute aufzeichnen und erst entschlüsseln, sobald ein ausreichend starker Quantencomputer verfügbar ist.

Hybrid-Handshake

Kombination aus klassischem und Post-Quantum-Verfahren im selben Schlüsselaustausch — bleibt sicher, solange mindestens eines der beiden Verfahren hält.

Tor / Hidden Service

Anonymisierungsnetzwerk, das Verbindungen über mehrere Relais verschlüsselt; ein Hidden Service macht einen Dienst nur über eine „onion“-Adresse erreichbar, ohne öffentliche IP/Domain.

One-Time-Pad (OTP)

Theoretisch perfekt sicheres Verschlüsselungsverfahren, das für jede Nachricht einen gleich langen, echt zufälligen und nur einmal verwendeten Schlüssel benötigt — in der Praxis wegen des Schlüsselaustausch-Aufwands kaum handhabbar.

CSPRNG

Kryptografisch sicherer Pseudozufallsgenerator (z. B. `crypto.getRandomValues()`) — der in der Praxis korrekte Standardansatz für Schlüsselmaterial, auch ohne spezielle Quantenhardware.